CLASSIFICATION ASSIGNMENT

1) Identify your problem statement

The goal of this problem is to build a predictive model that estimates whether a patient has **Chronic Kidney Disease (CKD) in classification column** (output/target) based on multiple medical parameters such as **Age, BP, SG, AL, SU, RBC, PC ,PCC, BA, BGR, BU, SC, SOD, POT, HRMO, PCV, WC, RC, HTN, DM, CAD, APPET, PE, ANE**, and other clinical attributes available in the dataset.

- Input and output data is clearly mentioned in this dataset. So this is **supervised learning.**
- Check the output variable type. Here output variable column is **Classification column**. This is **categories or label column** in the dataset like (**yes or no, 1 or 0)**. In Classification column **CKD** is yes or no values in this sheet. This is binary numbers is coming here. So this is **Classification.**

2) Tell basic info about the dataset (Total number of rows, columns

From the provided dataset like:

- Total Rows - 399
- Total Columns – 25

Column Details:

- Age, bp, al,su,bgr,bu,sc,sod,pot,hrmo,pcv,wc,rc - Numerical
- SG– Categorical(Ordinal)
- Rbc, pc, pcc, ba, htn, dm, cad, appet, pe, ane - Categorical
- Classification – Categorical binary (output variable)

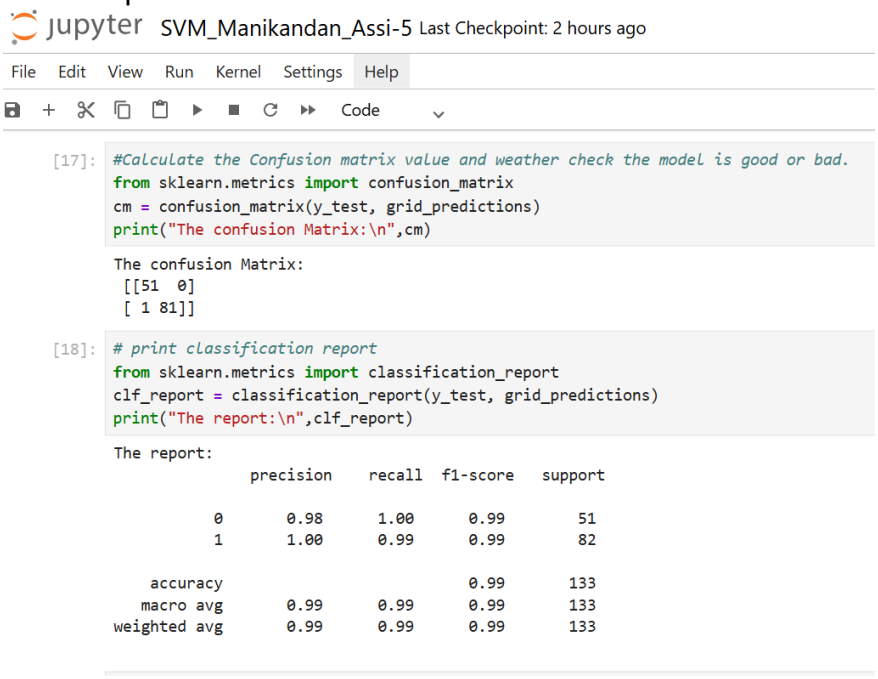
3) Data preprocessing steps

The data preprocessing steps needed for categorical data like **Rbc, pc, pcc, ba, htn, dm, cad, appet, pe, ane, SG, Classification** column. Here, convert nominal data into numerical data. This conversion can be done in **one-hot encoding**. For example

- PC-> encode as normal = 0 & abnormal = 1.
- PCC-> encode as present = 0 & notpresent = 1.
- htn-> encode as yes = 0 & no = 1.
- PCC-> encode as yes = 0 & poor = 1.
- Split the dataset through **training** & **test set**.

4) **Develop a good model with good evaluation metric. You can use any machine learning algorithm; you can create many models. Finally, you have to come up with final model**.
    - SVM Accuracy (F1 score) = **0.99**
    - Random forest Accuracy (F1 score) = **0.98**
    - Decision tree Accuracy (F1 score) = **0.95**
    - Logistic classification Accuracy (F1 score) = **0.93**
    - KNN Accuracy (F1 score) = **0.71**
    - Naivee bayee GaussianNB Accuracy (F1 score) = **0.98**
    - Naivee bayee MultinomialNB Accuracy (F1 score) = **0.81**
    - Naivee bayee BernoulliNB Accuracy (F1 score) = **0.93**
    - Naivee bayee ComplementNB Accuracy (F1 score) = **0.81**

5) **All the research values of each algorithm should be documented. (You can make tabulation or screenshot of the results.)**
    - SVM output



- Random Forest output

jupyter RF_Manikandan_Assi-5 Last Checkpoint: 10 minutes ago

File Edit View Run Kernel Settings Help

Code

[14]:
```
#Calculate the Confusion matrix value and weather check the model is good or bad
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, grid_predictions)
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[50  1]
 [ 1 81]]
```

[15]:
```
# print classification report
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)
print("The report:\n",clf_report)
```

```
The report:
              precision    recall  f1-score   support

           0       0.98      0.98      0.98        51
           1       0.99      0.99      0.99        82

    accuracy                           0.98       133
   macro avg       0.98      0.98      0.98       133
weighted avg       0.98      0.98      0.98       133
```

- Decision Tree output

jupyter DC_Manikandan_Assi-5 Last Checkpoint: 9 minutes ago

File Edit View Run Kernel Settings Help

Code

[14]:
```
#Calculate the Confusion matrix value and weather check the model is good or bad.
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, grid_predictions)
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
 [[51  0]
 [ 6 76]]
```

[15]:
```
# print classification report
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)
print("The report:\n",clf_report)
```

```
The report:
              precision    recall  f1-score   support

           0       0.89      1.00      0.94        51
           1       1.00      0.93      0.96        82

    accuracy                           0.95       133
   macro avg       0.95      0.96      0.95       133
weighted avg       0.96      0.95      0.96       133
```

- Logistic classification output

jupyter LogisticClassification_Manikandan_Assi-5 Last Checkpoint: 11 minutes ago

File Edit View Run Kernel Settings Help

Code

[13]:
```
#Calculate the Confusion matrix value and weather check the model is good or bad
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[42  3]
 [ 6 69]]
```

[14]:
```
#classification report - depends on confusion matrix
from sklearn.metrics import classification_report
clf_report = classification_report(y_test,y_pred)
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.88      0.93      0.90        45
           1       0.96      0.92      0.94        75

    accuracy                           0.93       120
   macro avg       0.92      0.93      0.92       120
weighted avg       0.93      0.93      0.93       120
```

- KNN Output

jupyter KNN_Manikandan_Assi-5 Last Checkpoint: 19 minutes ago

File Edit View Run Kernel Settings Help

Code

[16]:
```
#Calculate the Confusion matrix value and weather check the model is good or bad.
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[37  8]
 [27 48]]
```

[17]:
```
#classification report - depends on confusion matrix
from sklearn.metrics import classification_report
clf_report = classification_report(y_test,y_pred)
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.58      0.82      0.68        45
           1       0.86      0.64      0.73        75

    accuracy                           0.71       120
   macro avg       0.72      0.73      0.71       120
weighted avg       0.75      0.71      0.71       120
```

- Naivee bayee GaussianNB output

jupyter Naivebayes_Manikandan_Assi-5 Last Checkpoint: 25 minutes ago

File Edit View Run Kernel Settings Help

Code

```
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier = classifier.fit(X_train,y_train) # This code will create model
y_pred = classifier.predict(X_test) #Evaluation metrics - process of prediction
#Calculate the Confusion matrix value and weather check the model is good or bad.
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
#classification report - depends on confusion matrix
from sklearn.metrics import classification_report
clf_report = classification_report(y_test,y_pred)
print("-----------CLF Report - Start----------")
print(clf_report)
print("-----------CLF Report - End----------")
print(cm)
```

```
-----------CLF Report - Start----------
              precision    recall  f1-score   support

           0       0.96      1.00      0.98        45
           1       1.00      0.97      0.99        75

    accuracy                           0.98       120
   macro avg       0.98      0.99      0.98       120
weighted avg       0.98      0.98      0.98       120

-----------CLF Report - End----------
```

- Naivee bayee MultinomialNB output

jupyter Naivebayes_Manikandan_Assi-5 Last Checkpoint: 27 minutes ago

File Edit View Run Kernel Settings Help

Code

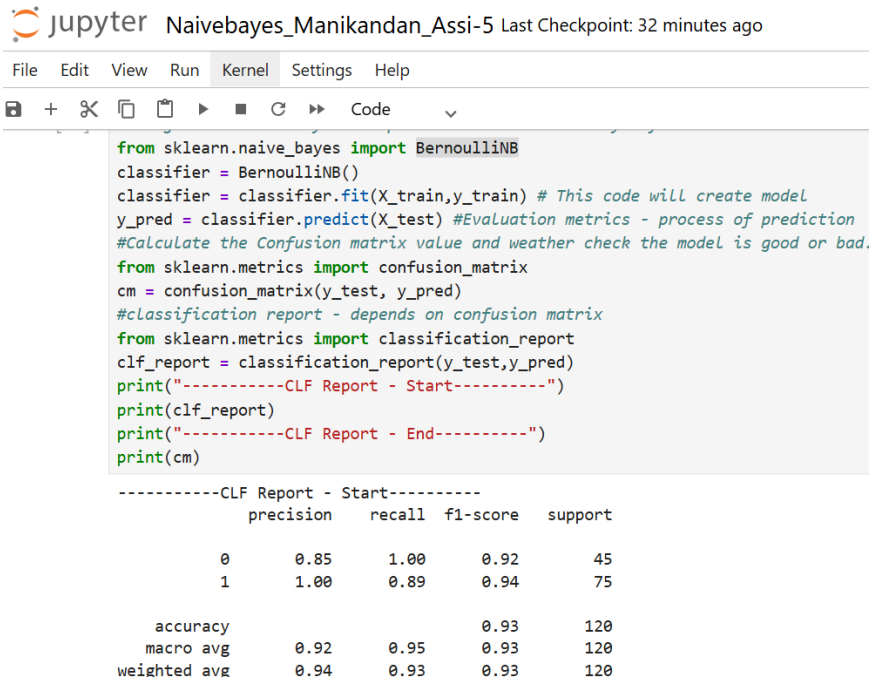
```
from sklearn.naive_bayes import MultinomialNB
classifier = MultinomialNB()
classifier = classifier.fit(X_train,y_train) # This code will create model
y_pred = classifier.predict(X_test) #Evaluation metrics - process of prediction
#Calculate the Confusion matrix value and weather check the model is good or bad.
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
#classification report - depends on confusion matrix
from sklearn.metrics import classification_report
clf_report = classification_report(y_test,y_pred)
print("-----------CLF Report - Start----------")
print(clf_report)
print("-----------CLF Report - End----------")
print(cm)
```

```
-----------CLF Report - Start----------
              precision    recall  f1-score   support

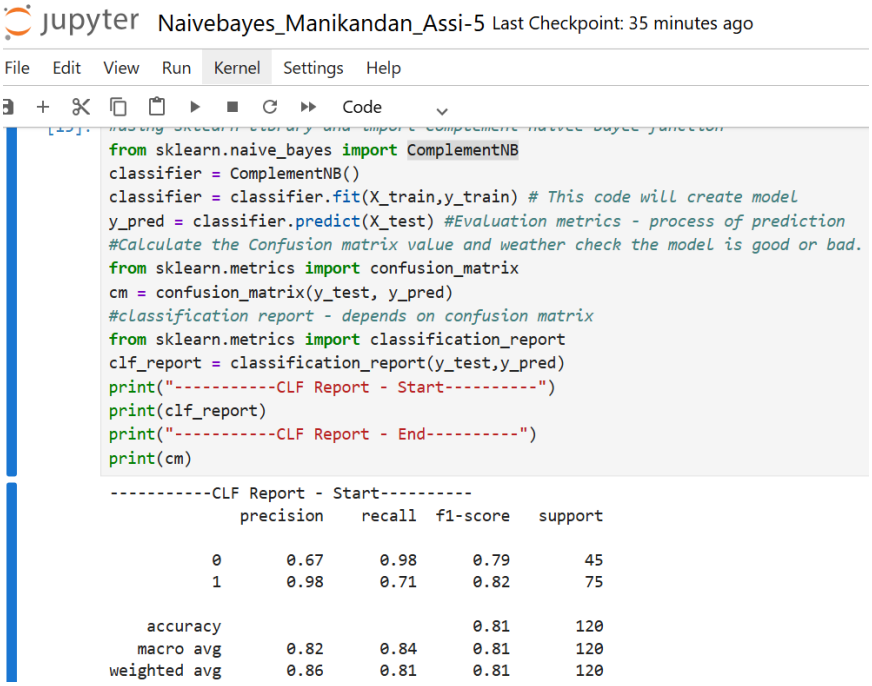
           0       0.67      0.98      0.79        45
           1       0.98      0.71      0.82        75

    accuracy                           0.81       120
   macro avg       0.82      0.84      0.81       120
weighted avg       0.86      0.81      0.81       120
```

- Naivee bayee BernoulliNB output



- Naivee bayee ComplementNB output



6) Mention your final model, justify why u have chosen the same.

The **SVM Classification** Accuracy(F1 Score) = 0.99. This is the best classification model. It provides the excellent balance between **Precision** and **Recall** and performs well on binary classification problem.